Today's agenda

- factorial
- ${}^nC_r$ & ${}^nP_r$
- functions

AlgoPrep

Q) factorial

↳ Given N, Print factorial of N.

Quiz 1: fact(4): 1*2*3*4 = 24

fact(N) = 1*2*3*4*.....*N

//Psuedo code

```
Public static void main() {
    Scanner scn = new Scanner(system.in);
    int n = scn.nextInt();

    int ans = 1;
    for (int i=1; i<=n; i++) {
        ans = ans * i;
    }
    System.out.println(ans);
}
```

AlgoPrep

```
int ans = 1;
for (int i = 1; i <= n; i++) {
    ans = ans * i;
}
System.out.println(ans);
```

n = 4

ans = ~~1 2 6~~ 24

i	i<=n
1	t
2	t
3	t
4	t
5	f

↳ exit

AlgoPrep

// ${}^nC_r$ and ${}^nP_r$

Quiz 2: ${}^5C_3 \rightarrow \frac{\lfloor 5}{\lfloor 3 * \lfloor 2} = \frac{120}{6*2} = 10$

Quiz 3: ${}^5P_3 \rightarrow \frac{\lfloor 5}{\lfloor 2} = \frac{120}{2} = 60$

${}^nC_r = \frac{\lfloor n}{\lfloor r \, \lfloor n-r}$

${}^nP_r = \frac{\lfloor n}{\lfloor n-r}$

Q) Given n and r, write an algorithm to calculate ${}^nC_r$.

$\frac{\lfloor n}{\lfloor r \, \lfloor n-r}$

AlgoPrep

//Psuedo code

```
P S v main ( ) {
    Scanner scn = new Scanner(system.in);
        int n = scn.nextInt();
        int r = scn.nextInt();

            int nfact = 1;
            for (int i = 1; i <= n ; i++) {
                    nfact = nfact * i;

            }

            int rfact = 1;
            for (int i = 1; i <= r ; i++ ) {
                    rfact = rfact * i;
            }

            int nmrfact = 1;
            for (int i = 1; i <= n-r; i++) {
                    nmrfact = nmrfact * i;
            }
        int ans = nfact / (rfact * nmrfact);

        System.out.Println(ans);
}
```

AlgoPrep

DRY → Do not repeat yourself

↓

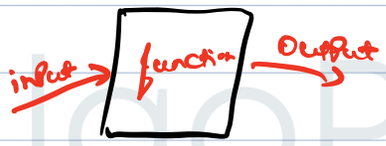
function

↳ Screw driver → open & close nut & bolt.

⇓

buy screwdriver

↳ open & close table's nut & bolt → use the previously bought screw driver.



Syntax:

skip the meaning for now

output type

function name

```
Public static int name ( input ) {

    //Statement 1
    // Statement 2

}
```

```
Public static void main() {
    Scanner scn = new Scanner(system.in);
    int n = scn.nextInt();

    int ans = 1;
    for (int i = 1; i <= n; i++) {
        ans = ans * i;
    }
    System.out.println(ans);
}
```

```
Public static void main() {
    Scanner scn = new Scanner(system.in);
    int n = scn.nextInt();
    fact(n);
}

public static int fact(int n){
    int ans = 1;
    for (int i = 1; i <= n; i++) {
        ans = ans * i;
    }
    return ans;
}
```

```
Public static void main() {
    Scanner scn = new Scanner(system.in);
    int n = scn.nextInt();
    int n1 = fact(n);
    System.out.Println(n1);
}

public static int fact(int n){
    int ans = 1;
    for (int i = 1; i <= n; i++) {
        ans = ans * i;
    }
    return ans;
}
```

24

Call stack

fact { ~~ans = 1*2*3*4 = 24~~ ~~n = 4~~

main { n1 = 24, n = 4, scn = ✓

24

return in function == Break in for loop.

→ Called funct() but you don't want to return anything from there → type of function should be void.

Break till 9:23 Pm

// ^nC_r using function

```
Public static void main() {
    Scanner scn = new Scanner(system.in);
    int n = scn.nextInt();
    int r = scn.nextInt();

    int nfact = fact(n);
    int rfact = fact(r);
    int nmrfact = fact(n-r);

    int ans = nfact / (rfact * nmrfact);
    System.out.println(ans);

}
```

```
public static int fact(int n){
    int ans = 1;
    for(int i=1; i<=n; i++){
        ans = ans * i;

    }
    return ans;
}
```

AlgoPrep

^nC_r

Call Stack

```
Public static void main() {
    Scanner scn = new Scanner(System.in);
    int n = scn.nextInt();
    int r = scn.nextInt();

    int nfact = fact(n);
    int rfact = fact(r);
    int nmrfact = fact(n-r);

    int ans = nfact / (rfact * nmrfact);
    System.out.println(ans);
}
```

↳ 10

```
public static int fact(int n){
    int ans = 1;
    for (int i=1; i<=n; i++) {
        ans = ans * i;
    }
    return ans;
}
```

~~fact~~	~~ans = 1*1*2 = 2~~ ~~n = 2~~
~~fact~~	~~ans = 1*1*2*3 = 6~~ ~~n = 3~~
~~fact~~	~~ans = 1*1*2*3*4*5 = 120~~ ~~n = 5~~
main	nmrfact = 2 rfact = 6 nfact = 120 r = 3 n = 5 scn = k

ans = $\frac{120}{12}$ = 10

(n-r ⇒ 5-3 = 2

```
→ main() {
    int ans = sum(10, 20, 30);
}
```

← error

sum(10, 20, 30)

```
Public static void sum (int a, int b, int c){
    return a+b+c;
}
```

int

10 20 30

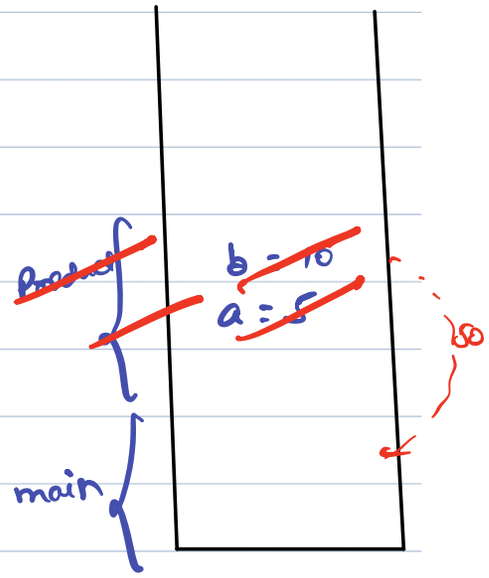
AlgoPrep

Quiz 4:

```
Public static void main (String [] args) {
        ~~Product (5,10);~~
                50
}
```

```
Public static int Product (int a, int b) {
        return a*b;
}
```

↳ no output



Quiz 5:

```
Public static void main (String [] args) {
        int ans = Subtract (5,10);
        System.out.println (ans);
}
```

→ error

```
Public static void Subtract (int a, int b) {
        return a - b;
}
```

AlgoPrep

Quiz 6:

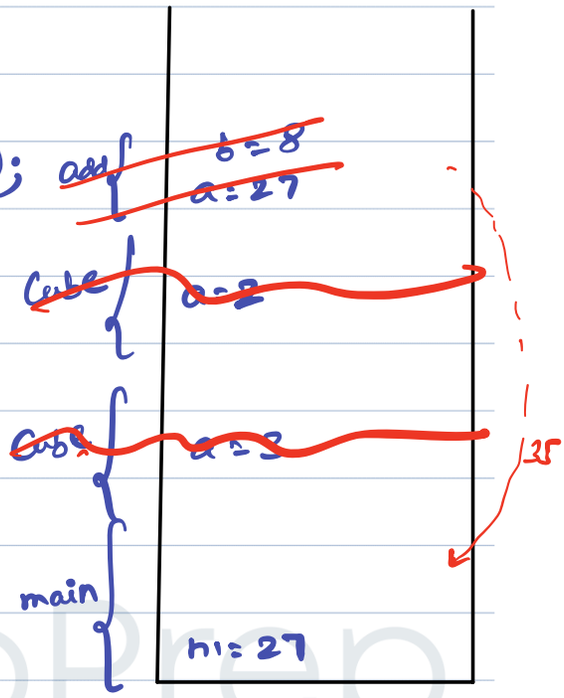
```
Public static void main (String[] args){
    int n1 = cube(3);
    System.out.println( add (n1, cube(2)) );
}
```

27 8

↳ 35

```
Public static int add (int a, int b){
        return a+b;
}
```

```
Public static int cube (int a){
    → return a*a*a;
}
```



AlgoPrep